

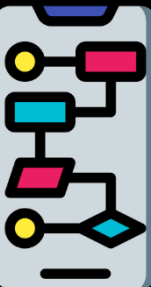
C Programming

14_linked_lists

Алгоритмы

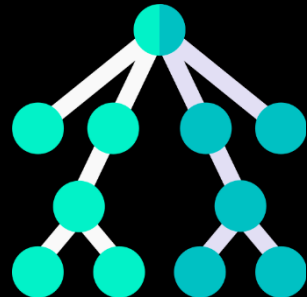
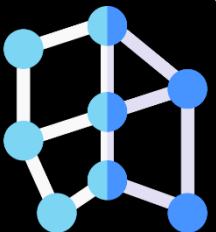
Алгоритм — есть последовательность действий (или инструкций) для выполнения задачи и получения некоторого результата. Зная алгоритм его легко можно перенести в программу написанную на **любом** языке, т.к. алгоритм от языка к языку остается прежним

- Алгоритмы повторяемы и выдают предсказуемые результаты каждый раз, когда они выполняются.
- Алгоритмы можно стандартизировать и совместно использовать скажем в разных устройствах. Пример: стандарт 4G на телефоне - по сути это программа, состоящая из набора алгоритмов, благодаря которым в какой стране вы бы не оказались вам не придется менять телефон, чтобы пользоваться мобильной связью
- Алгоритм не зависит от используемого языка. Он сообщает программисту логику, используемую для решения проблемы. Таким образом, алгоритм - это логическая пошаговая процедура, которая служит программистам своего рода планом.



Типы алгоритмов

- Алгоритмы сортировки - алгоритмы для упорядочивания каких-либо данных в определенном порядке. В интернет-магазине включаем фильтр по количеству отзывов. Пузырьковая сортировка, сортировка вставками, быстрая сортировка, сортировка подсчетом и много-много других..
- Алгоритмы поиска - алгоритмы поиска значения в наборе данных. Линейный поиск, бинарный и т.д.
- Алгоритмы с графами - поиск кратчайших путей. Алгоритм Дейкстры, BFS, DFS и т.д. Пример в 2gis ищем как пройти от какого-нибудь отеля до ТЦ. Социальные сети работают на алгоритмах с графами



Зачем алгоритмы?

Кол-во элементов (N)	Линейный поиск ($O(N)$)	Бинарный поиск ($O(\log N)$)	Разница (эффективность)
100	100 мс	~7 мс	в 14 раз быстрее
10 000	10 секунд	~14 мс	в 700 раз быстрее
1 000 000	~11.5 дней	~20 мс	в 50 000 000 раз быстрее
1 000 000 000	~31.7 год(а)	~30 мс	в 33 млрд раз быстрее
(Атомы)	> времени жизни Вселенной	~265 мс	Мгновенно vs Никогда

Сложность алгоритмов

Сложность в алгоритмах относится к количеству ресурсов, необходимых для решения проблемы или выполнения задачи. К таким ресурсам относят время и память. (Time and Space complexity)

- Память (Space Complexity) - сколько оперативной памяти занимает программа, переменные и т.д.
- Время (Time Complexity) - сколько времени нужно алгоритму на выполнение. Эта мера показывает зависимость между временем выполнения и кол-вом входных данных



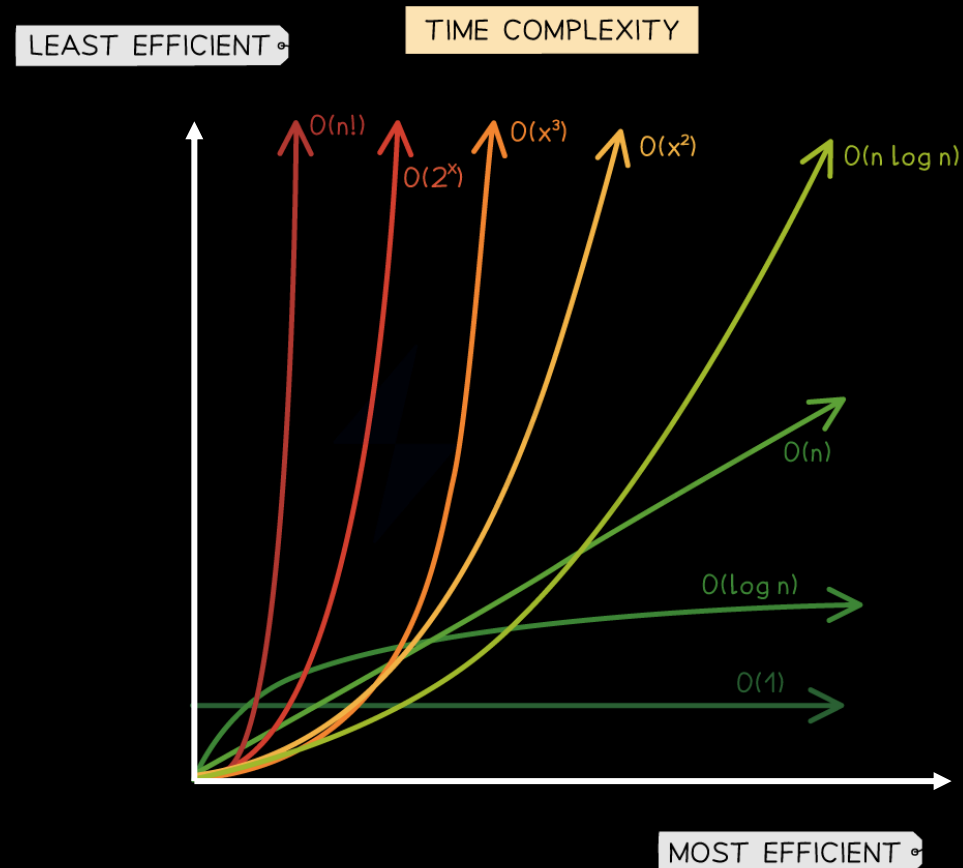
Big O notation

Big O нотация используется для оценки наихудшего случая временной сложности алгоритма, т.е. "O" покажет как поменяется производительность алгоритма.

Записывается обычно следующим образом: $O(n)$ - читается как "O от n", где n - размер входных данных.

Сложность:

- $O(1)$ - Константная ~
- $O(\log n)$ - Логарифмическая ~
- $O(n)$ - Линейная ~
- $O(n \log n)$ - Линейно-логарифмическая ~
- $O(n^2)$ - Квадратичная ~
- $O(n^3)$ - Кубическая ~
- $O(2^n)$ - Экспоненциальная ~
- $O(n!)$ - Факториальная ~

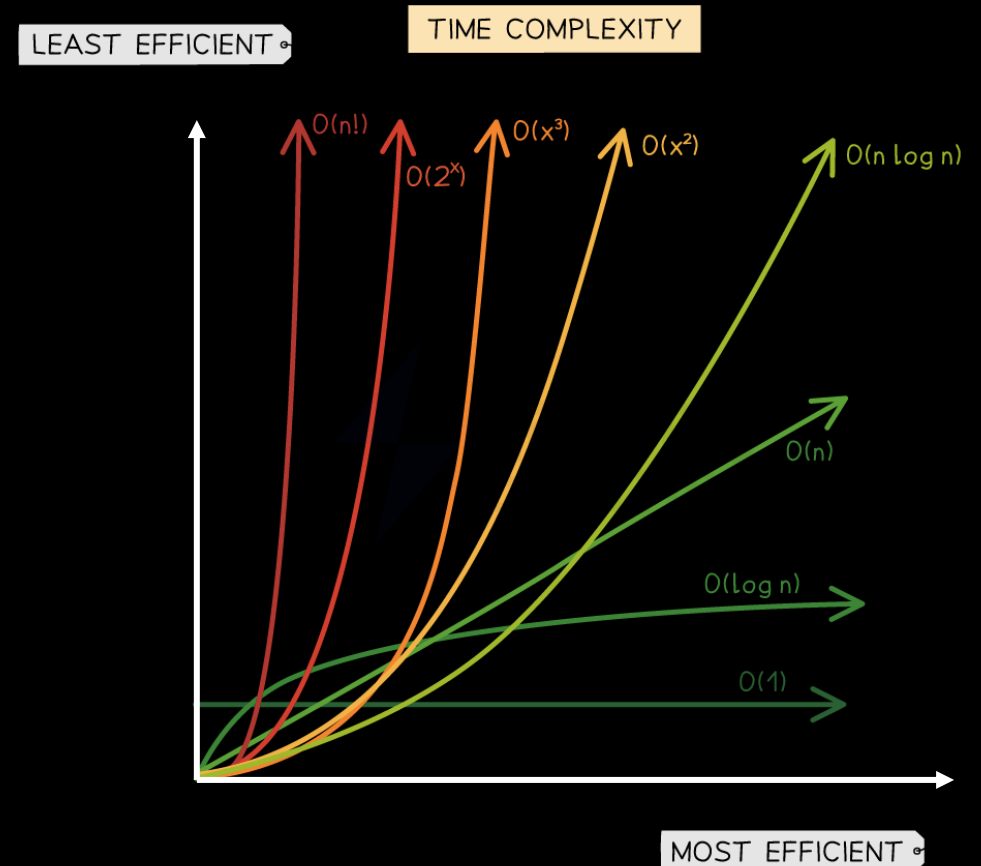


$O(1)$ Константная сложность

Такая сложность можно сказать является идеальной, т.к. производительность алгоритмов с такой сложностью не зависит от размера входных данных. Популярный пример такой сложности - получение элемента массива.

```
int some_int_array[1000];

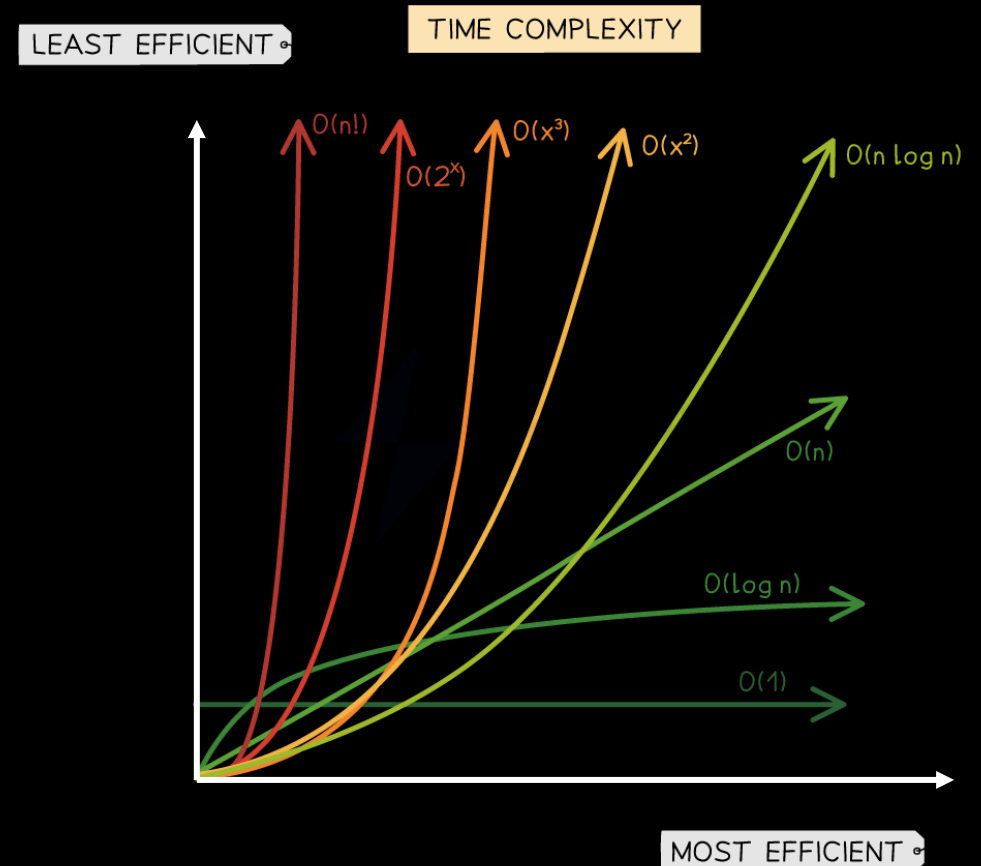
int get_elem(int idx) {
    return some_int_array[idx];
}
```



$O(n)$ Линейная сложность



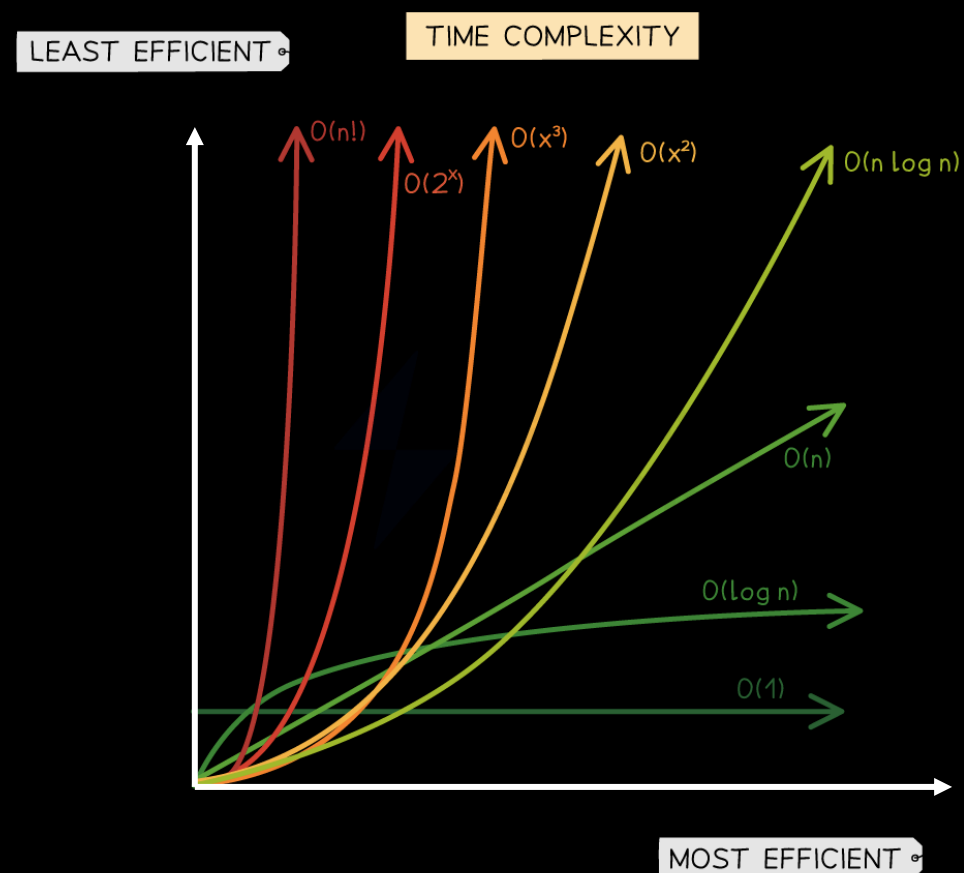
```
for (int s = 0; s < n; s++) {  
    if (recipes[s] == 'M'){  
        // get recipe  
    }  
}
```



$O(\log n)$ Логарифмическая сложность

1. Поделим алфавит на 2 части А-Я: А-О и П-Я -> буква М в первой половине алфавита, соответственно вторая половина нам не нужна
 2. Делим первую половину еще на две половины А-О: А-Ж и З-О -> буква М во второй половине, первую отбрасываем А-Ж
 3. З-О -> З-К и Л-О -> Л-О
 4. Л-О -> Л-М и Н-О -> Л-М
 5. Л и М -> М ответ
- 5 операций чтобы найти ответ, где $n = 33$, а $\log(33) = 5$

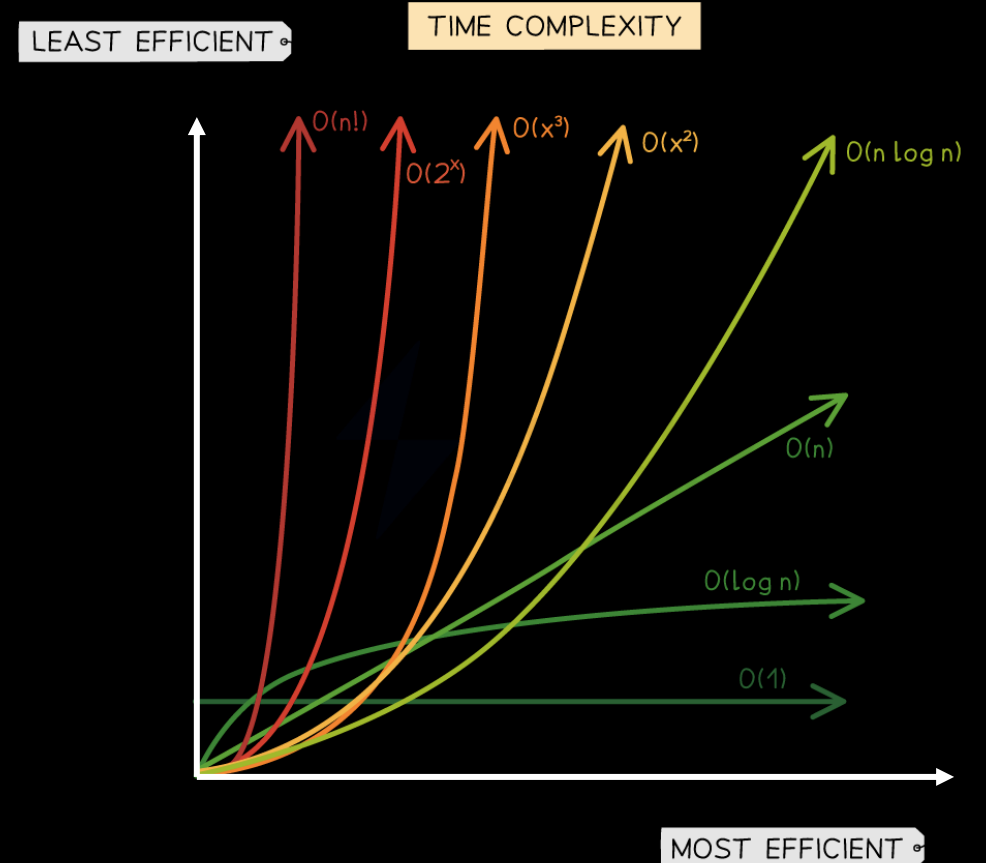
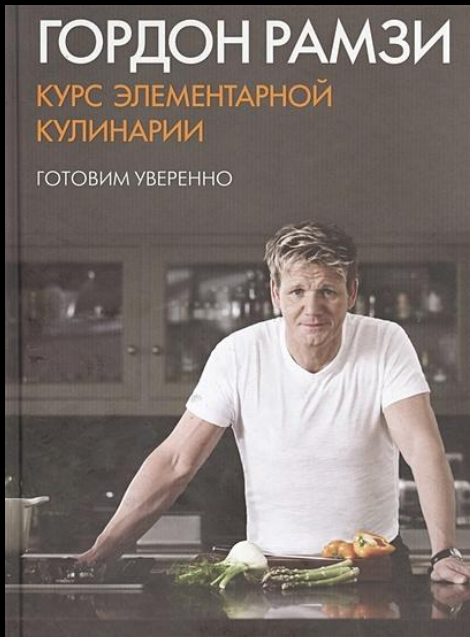
А	Б	В	Г	Д	Е	Ё	Ж	З	И	Й
1	2	3	4	5	6	7	8	9	10	11
К	Л	М	Н	О	П	Р	С	Т	У	Ф
12	13	14	15	16	17	18	19	20	21	22
Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
23	24	25	26	27	28	29	30	31	32	33



$O(n \log n)$ - Линейно-логарифмическая сложность

Она схожа с $O(\log n)$, но добавляется еще одна n - например мы ищем не в одной книге, а среди n книг рецептов Гордона Рамзи рецепт на какую-то букву.

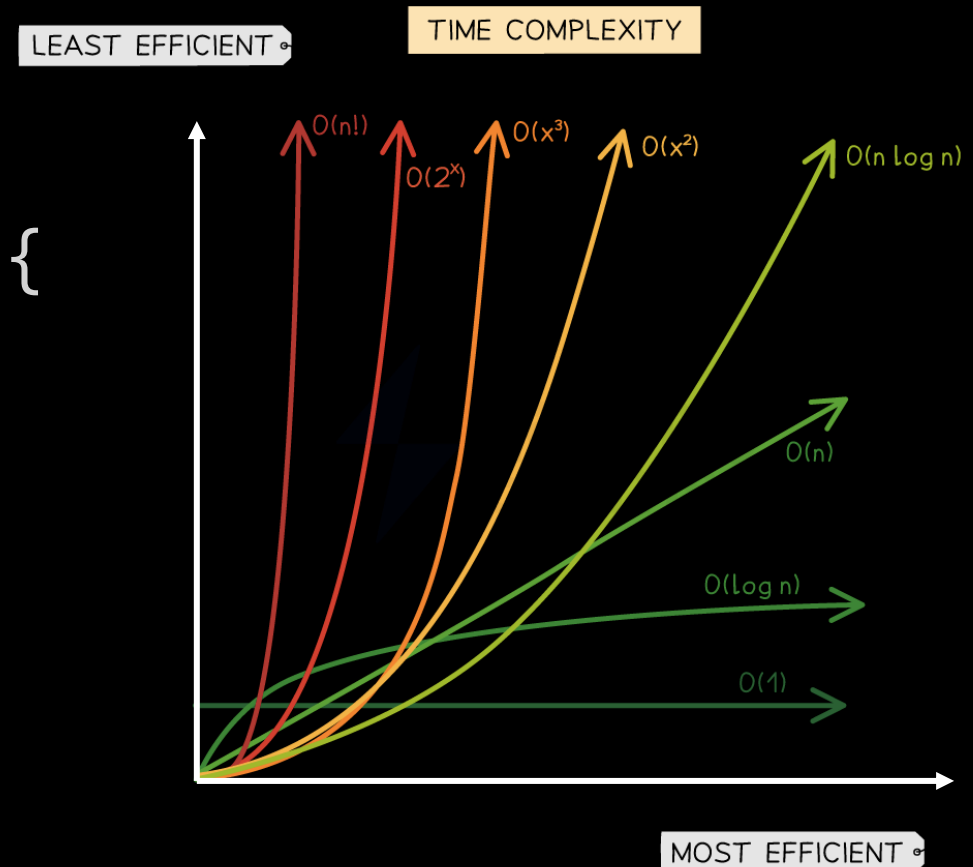
Итого: есть цикл $O(n)$, внутри которого используются алгоритмы "разделяй и властвуй" $O(\log n) \rightarrow O(n * \log n)$



$O(n^2)$ Квадратичная сложность

Сюда относятся все вложенные циклы, т.е. цикл в цикле. Самый простой пример - пузырьковая сортировка.

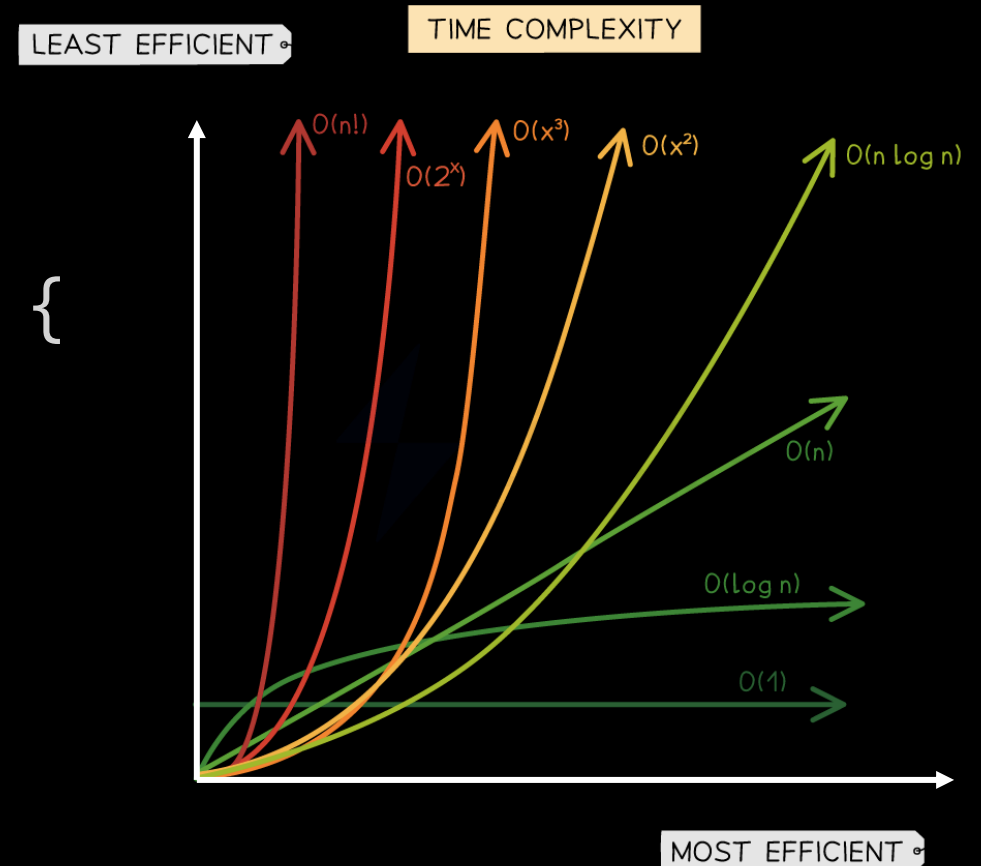
```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n - 1; j++) {  
        if (a[j] > a[j + 1]) {  
            swap(&a[j], &a[j + 1]);  
        }  
    }  
}
```



$O(n^3)$ Кубическая сложность

Если $O(n)$ - один проход по набору данных (один цикл), $O(n^2)$ - $n*n$ проходом по данным (два цикла - цикл в цикле), то $O(n^3)$ три цикла, пример:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        for (int k = 0; k < n; k++) {  
            // do smth  
        }  
    }  
}
```

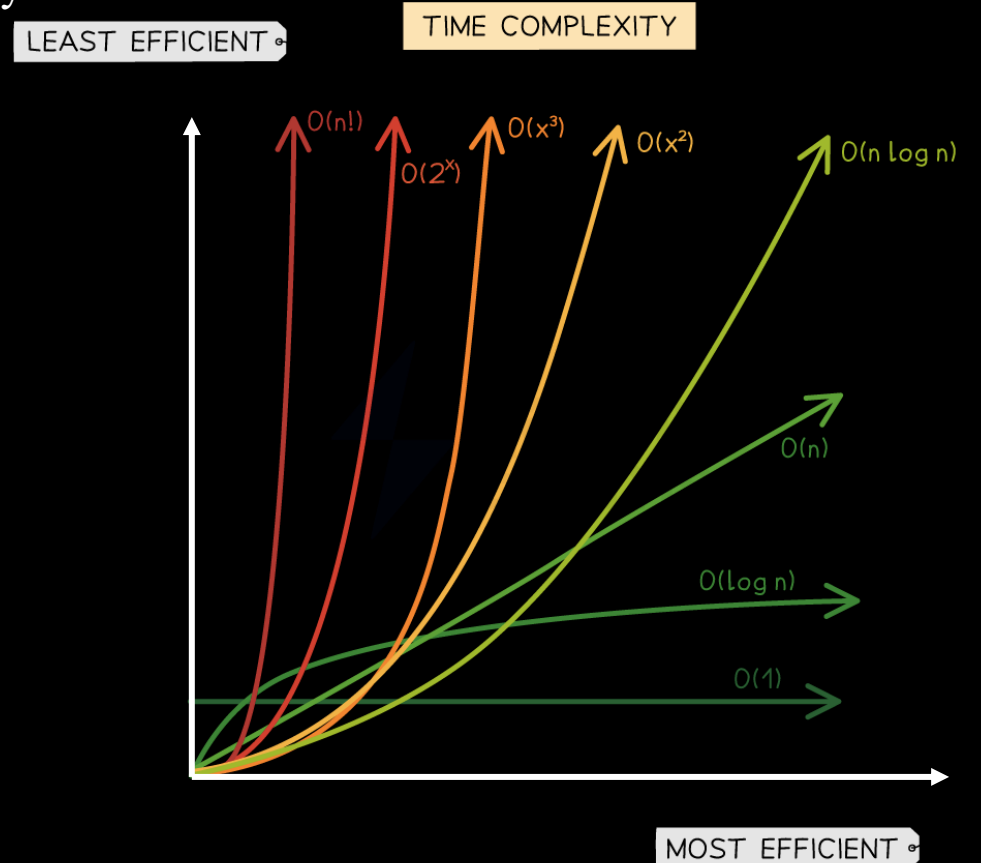


$O(2^n)$ Экспоненциальная сложность

Такую сложность можно встретить в рекурсиях, например при расчете чисел Фибоначчи. В данном коде используется рекурсия - функция, вызывающая саму себя. Однако каждый раз, когда вызывается функция `fib()`, она порождает за собой два дополнительных вызова, что приводит к экспоненциальному увеличению количества вызовов функций с увеличением n .

```
int fib(int n) {  
    if (n <= 1) {  
        return n;  
    } else {  
        return fib(n - 1) + fib(n - 2);  
    }  
}
```

```
fib(4) -> fib(3) + fib(2) = 5  
          |       |  
        fib(2) + fib(1) + fib(1) + fib(0)  
          |   |   |   |   |  
          1 + 1 + 1 + 1 + 1 = 5
```

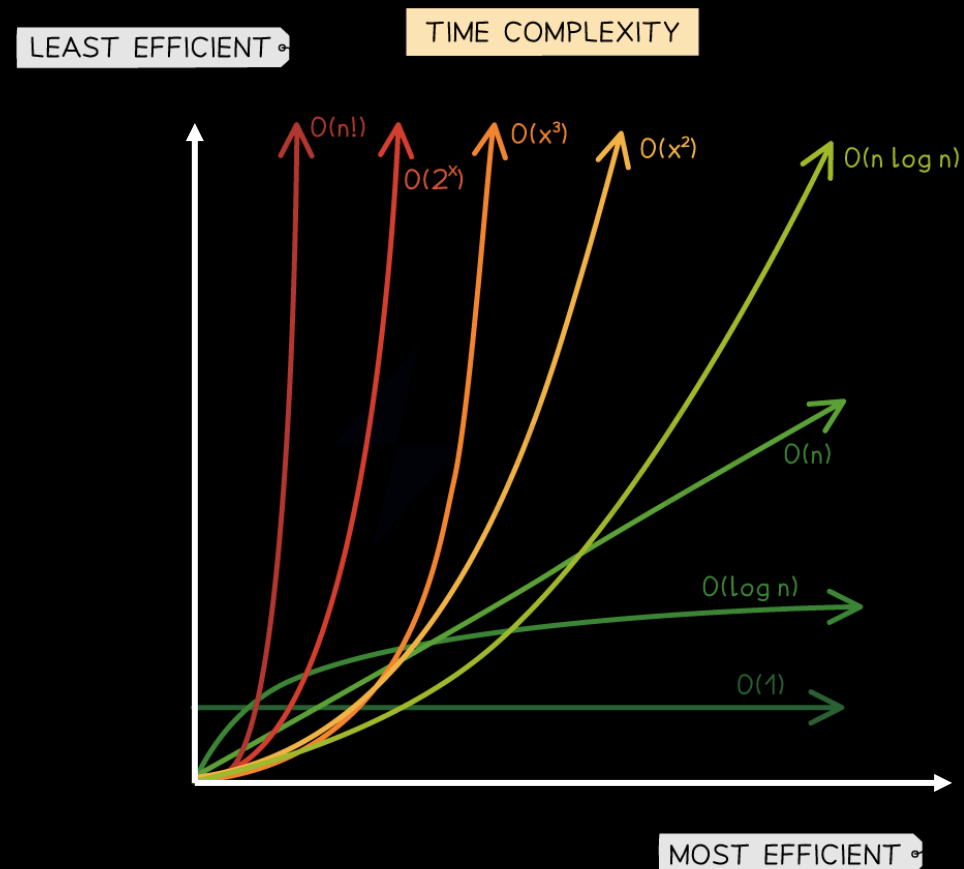


$O(n!)$ Факториальная сложность

Здесь время выполнения алгоритма растет пропорционально факториалу размера входных данных. Самый простой пример такой трудозатратной функции:

```
void f(int n) {  
    for(int i = 0; i < n; i++) {  
        f(n - 1);  
    }  
}
```

Вызов функции $f()$ от n единожды порождает n функций, вызывающих $n-1$ функций, вызывающих $n-2$ функций..



Нюансы Big O

В программировании абсолютно нормальна практика с упрощением букв и цифр в big O по возможности, например:

- $O(n) == O(2*n)$
- $O(n^2 + n) == O(n^2)$

$$O(n^2) + O(\log n) = O(n^2)$$

$$O(3n) = O(n)$$

$$O(10000 n^2) = O(n^2)$$

$$O(2n * \log n) = O(n * \log n)$$

$$O(n^2 + n) = O(n^2)$$

$$O(n^3 + 100n * \log n) = O(n^3)$$

$$O(n! + 999) = O(n!)$$

Пытаемся оценить сложность алгоритма

```
int findMax(int arr[], int length) {  
    int max = arr[0];  
    for (int i = 1; i < length; i++) {  
        if (arr[i] > max) {  
            max = arr[i];  
        }  
    }  
    return max;  
}
```


Пытаемся оценить сложность алгоритма

```
int sumOfPairs(int arr[], int length) {  
    int sum = 0;  
    for (int i = 0; i < length; i++) {  
        for (int j = 0; j < length; j++) {  
            sum += arr[i] + arr[j];  
        }  
    }  
    return sum;  
}
```

Пытаемся оценить сложность алгоритма

```
const int keys[] = {0x0, 0xDEADBEEF, 0xC0FFEE, 0xFFFF};
```

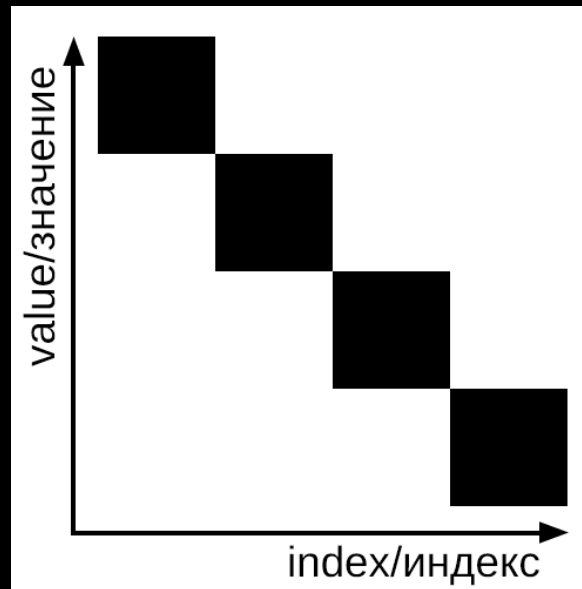
```
int get_key(int n) {  
    return keys[n];  
}
```

Пытаемся оценить сложность алгоритма

```
int test(int arr[], int length) {  
    int sum = 0;  
    for (int i = 0; i < length; i++) {  
        for (int j = 0; j < length; j++) {  
            sum += arr[i] + arr[j];  
        }  
    }  
    if (length < 3) return sum;  
    int max = arr[0];  
    for (int i = 1; i < 3; i++) {  
        if (arr[i] > max) {  
            max = arr[i];  
        }  
    }  
    return sum + max;  
}
```

Сортировки

Залазим на [wiki](#) с сортировками

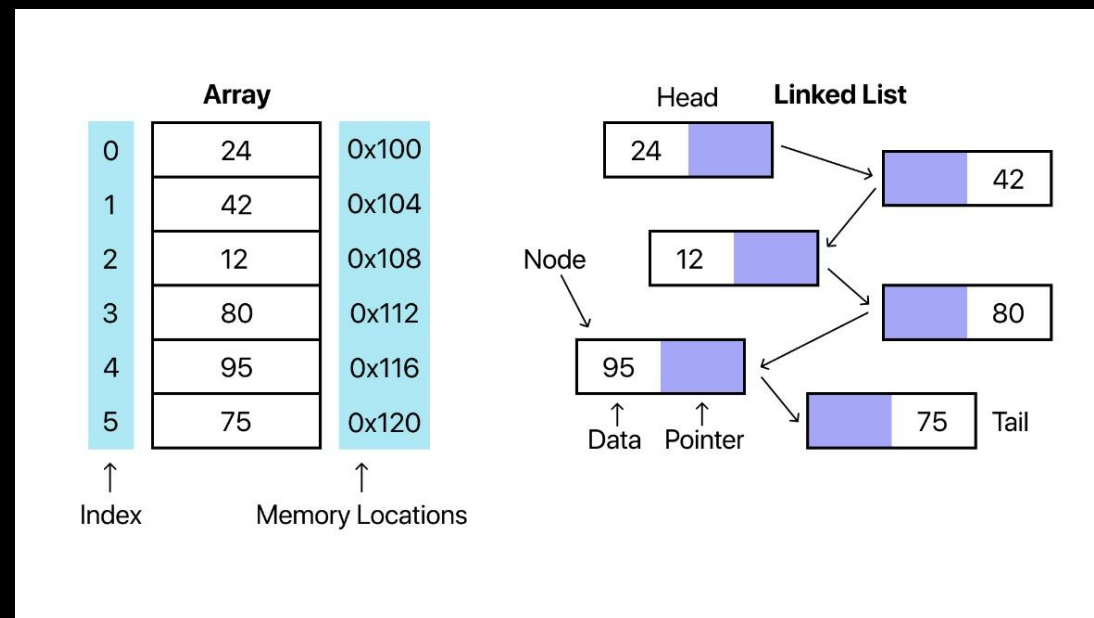


Связанные списки (Linked List)

Связанный список представляет собой последовательность узлов, где каждый узел состоит из двух компонентов:

1. Данные: любые значения, например, число, структуры и т.д.
2. Указатель: ссылка на следующий узел в списке.

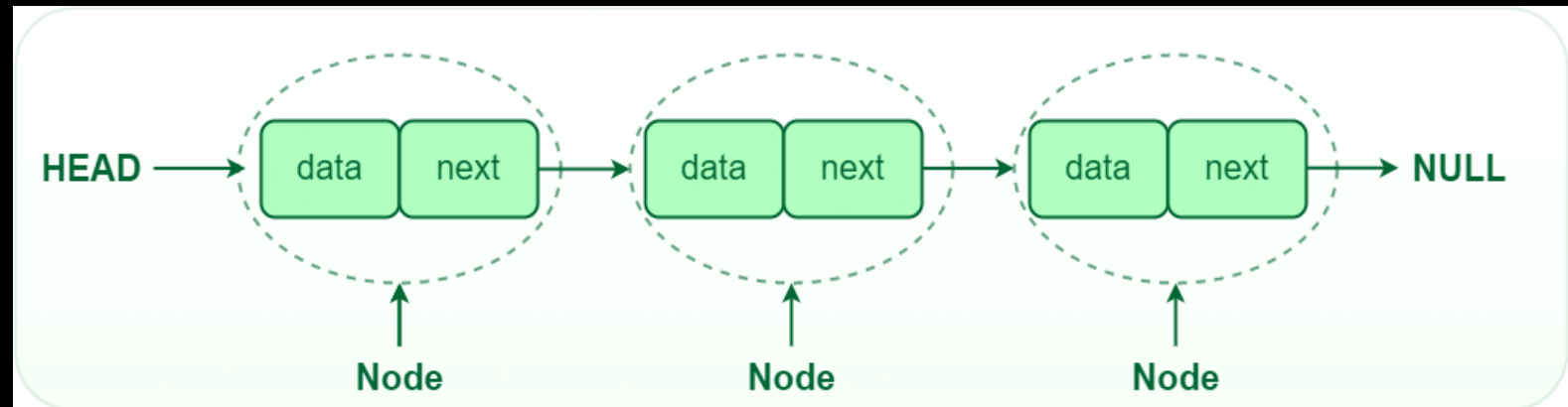
Чтобы получить доступ к любому элементу (узлу), необходимо последовательно пройти по всем узлам перед ним.



Односвязные списки

Односвязный список состоит из узлов. В каждом узле хранится указатель на следующий узел и сами данные. Указатели последнего узла (хвост - tail) указывают на NULL, что указывает на конец связанного списка.

```
struct node
{
    int data;
    struct node *next;
};
typedef struct node node_t;
```



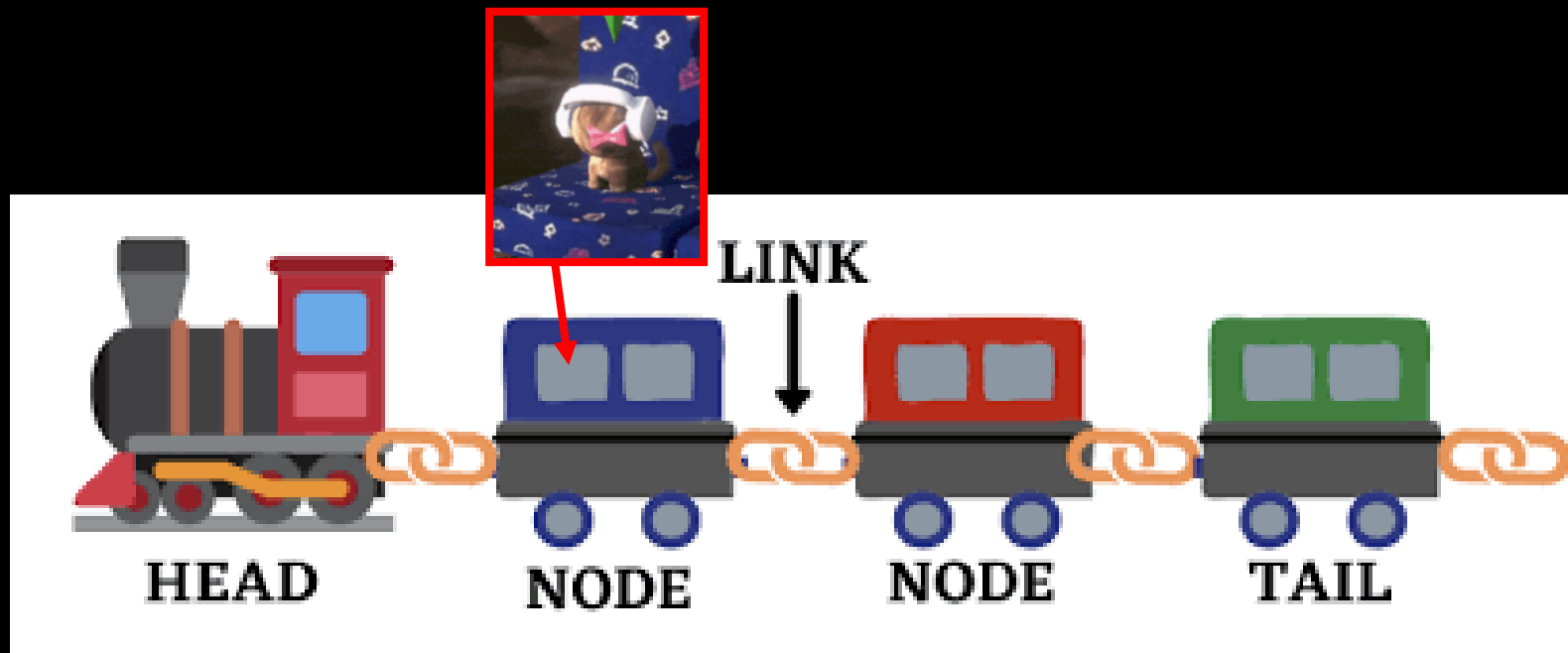
Связанный список

Связанный список — это структура данных, которая хранит элементы в линейном порядке и позволяет вставлять и удалять элементы в любом месте последовательности. Списки могут быть односвязными, двусвязными или круговыми (циклическими).

Параметр	Массив (статический)	Массив (динамический)	Односвязный список	Двусвязный список
Доступ	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Вставка/ Удаление в начало	--- (размер фиксированный)	$O(n)$ (нужно двигать элементы)	$O(1)$	$O(1)$
Вставка/ удаление в конец	--- (размер фиксированный)	$O(1)$ (если есть место в массиве), $O(n)$ (если места нет - копирование); удаление $O(1)$	$O(n)$	$O(1)$
Вставка/ удаление в середину	--- (размер фиксированный)	$O(n)$ (нужно двигать элементы)	$O(1)$ (но добираться до середины $O(n)$)	$O(1)$ (но добираться до середины $O(n)$)
Поиск	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Создадим свой односвязный список

Здесь начинаем live code (программировать в живую): Создадим односвязный список и напомним функции вставка/удаление, поиск, длина списка и вывод списка



Создаем узлы

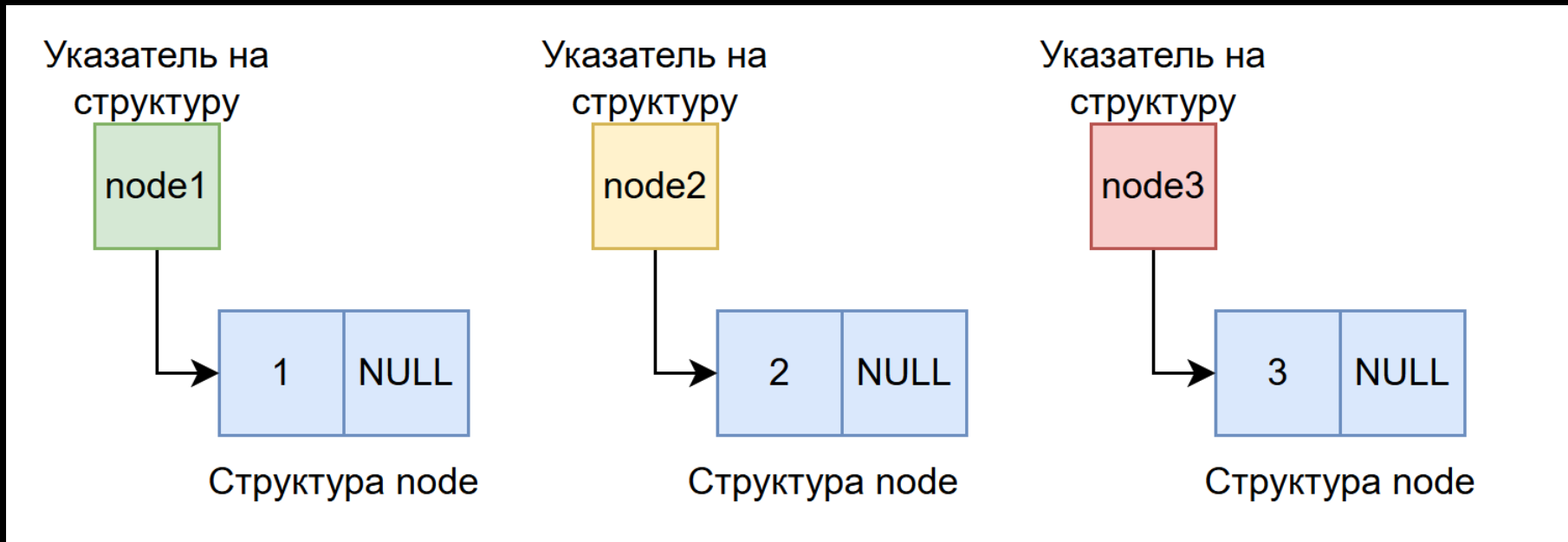
```
node_t *newNode(int data);
```

```
...
```

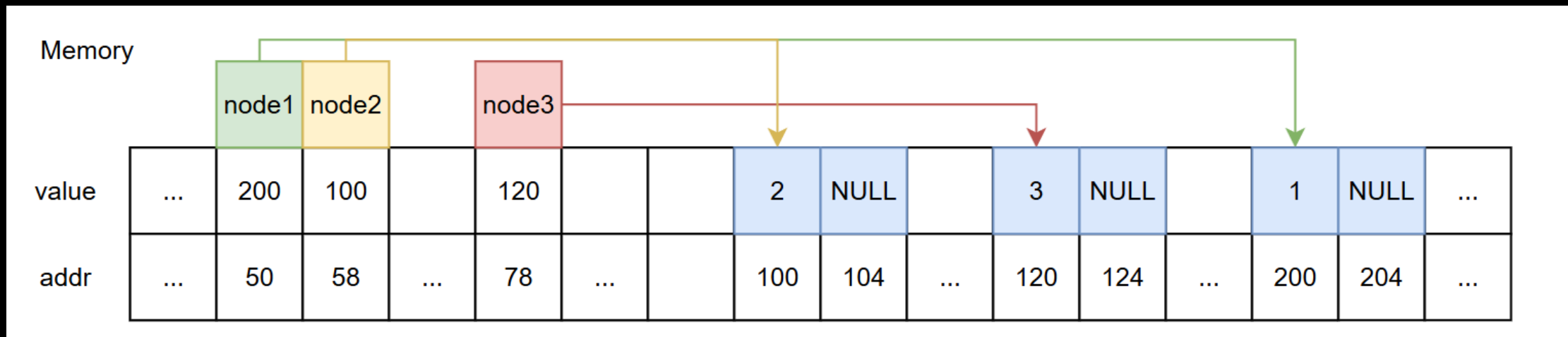
```
node_t *node1 = newNode(1);
```

```
node_t *node2 = newNode(2);
```

```
node_t *node3 = newNode(3);
```



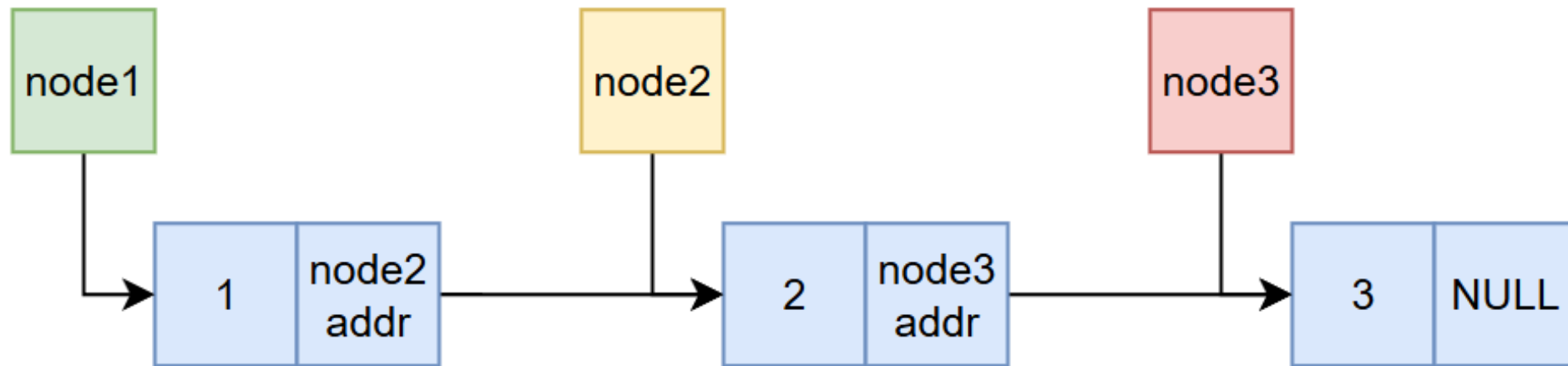
Создаем узлы



Мы выделили память под три структуры node, заполнили их числами и адрес начала каждой структуры записали в указатели node1, node2 и node3 соответственно

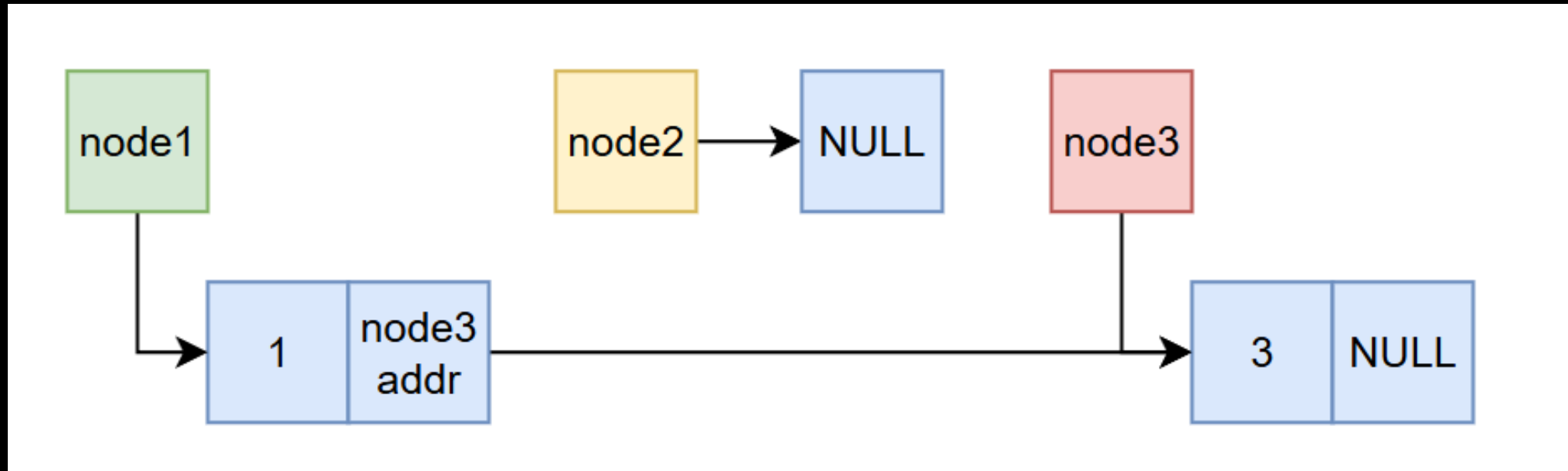
Указатели node1, node2 и node3 находятся в стеке и вероятнее всего будут лежать рядом друг с другом и друг за другом, а вот сами структуры находятся в куче и могут лежать совсем не по порядку

Связываем узлы



`node1->next = node2;` `node2->next = node3;`

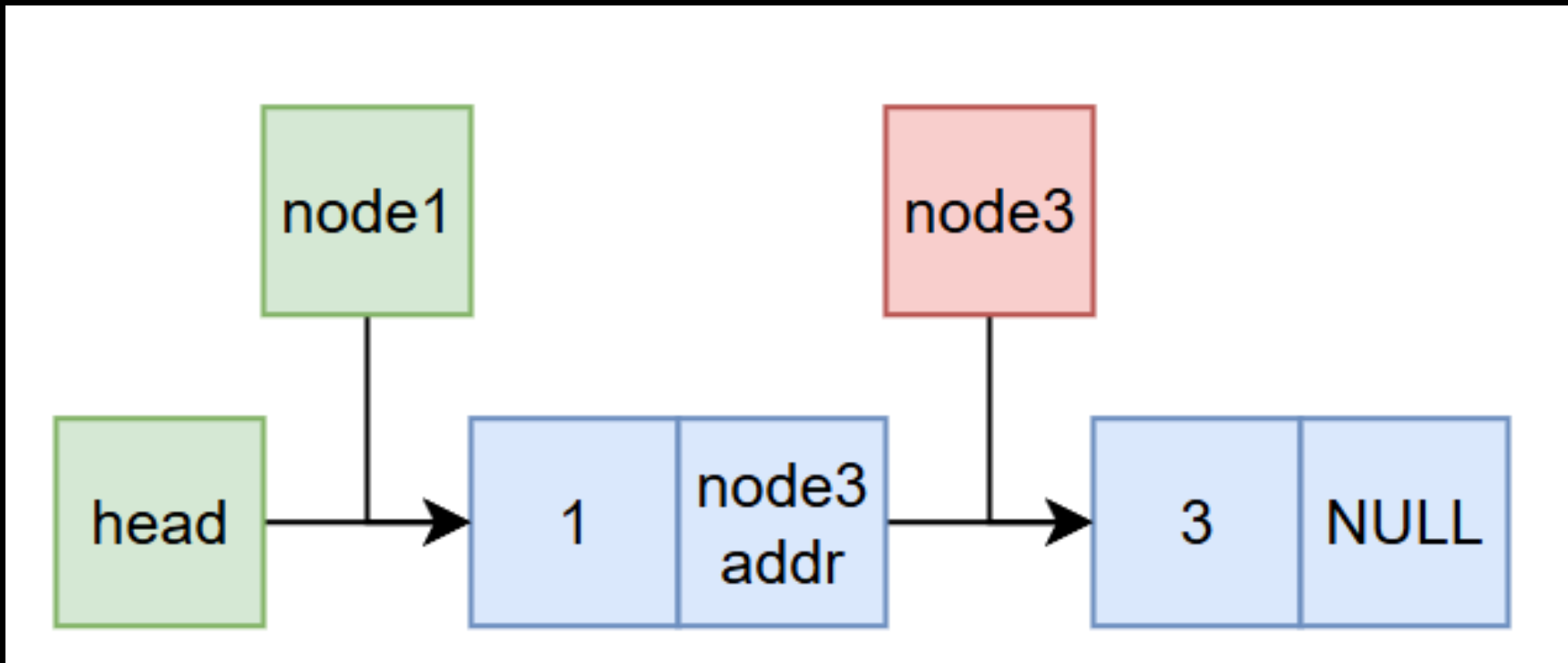
Развяжем узел 2 (delete node 2)



В создании списка таким образом есть нюанс: под каждый узел мы создавали свою переменную-указатель со своим именем, а это не серьезно если мы захотим расширяться. В реальности пишут отдельные функции для добавления узла в список, например в конец, начало, между двумя любыми (в середину).

Добавим голову к списку

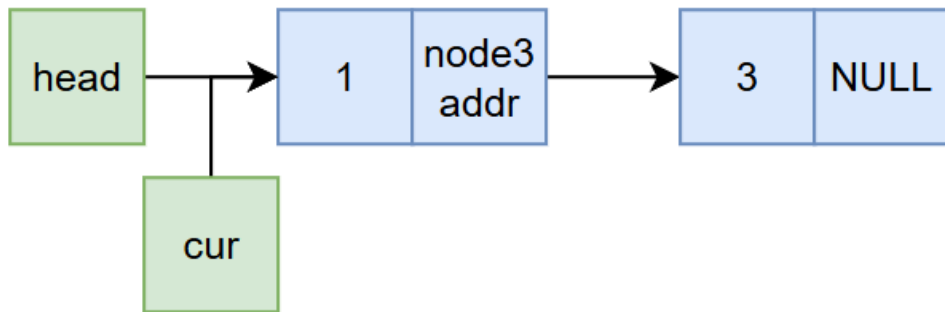
```
node_t* head = NULL; // Список пустой  
head = node1; // привяжемся к списку 1 -> 3
```



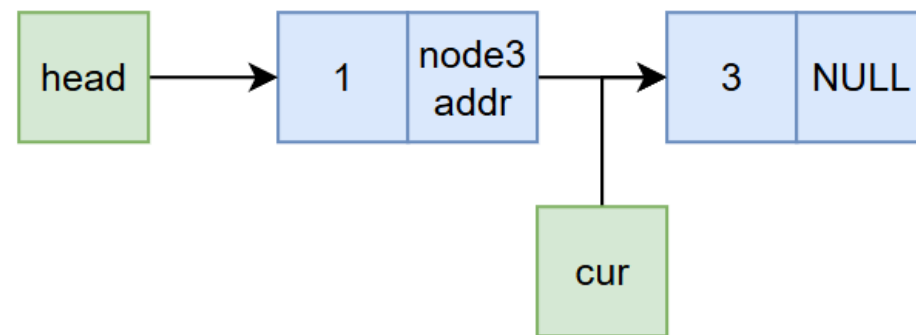
Проход по списку и его вывод

- Пока текущий узел не равен NULL:
 - Печатаем на экран данные
 - Приравниваем текущий указатель на следующий
- Список закончился

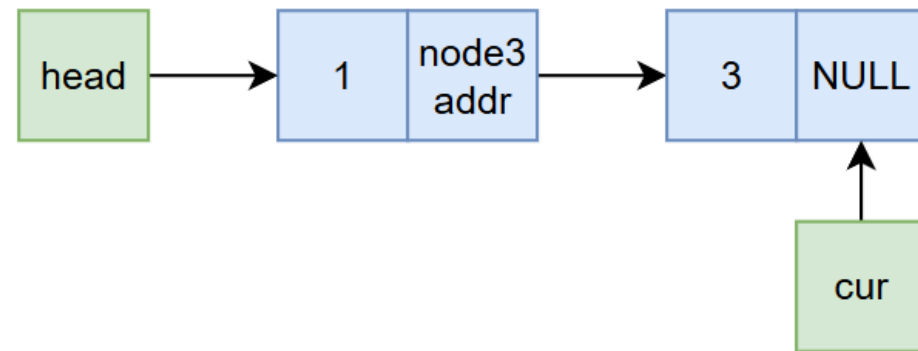
Итерация 1



Итерация 2



Итерация 3



Длина списка

```
// Возвращает длину списка (int)  
int lengthList(node_t* head);
```

- Длина списка = 0
- Пока текущий узел не равен NULL:
 - Длина списка + 1
 - Приравниваем текущий указатель на следующий
- Возвращаем длину

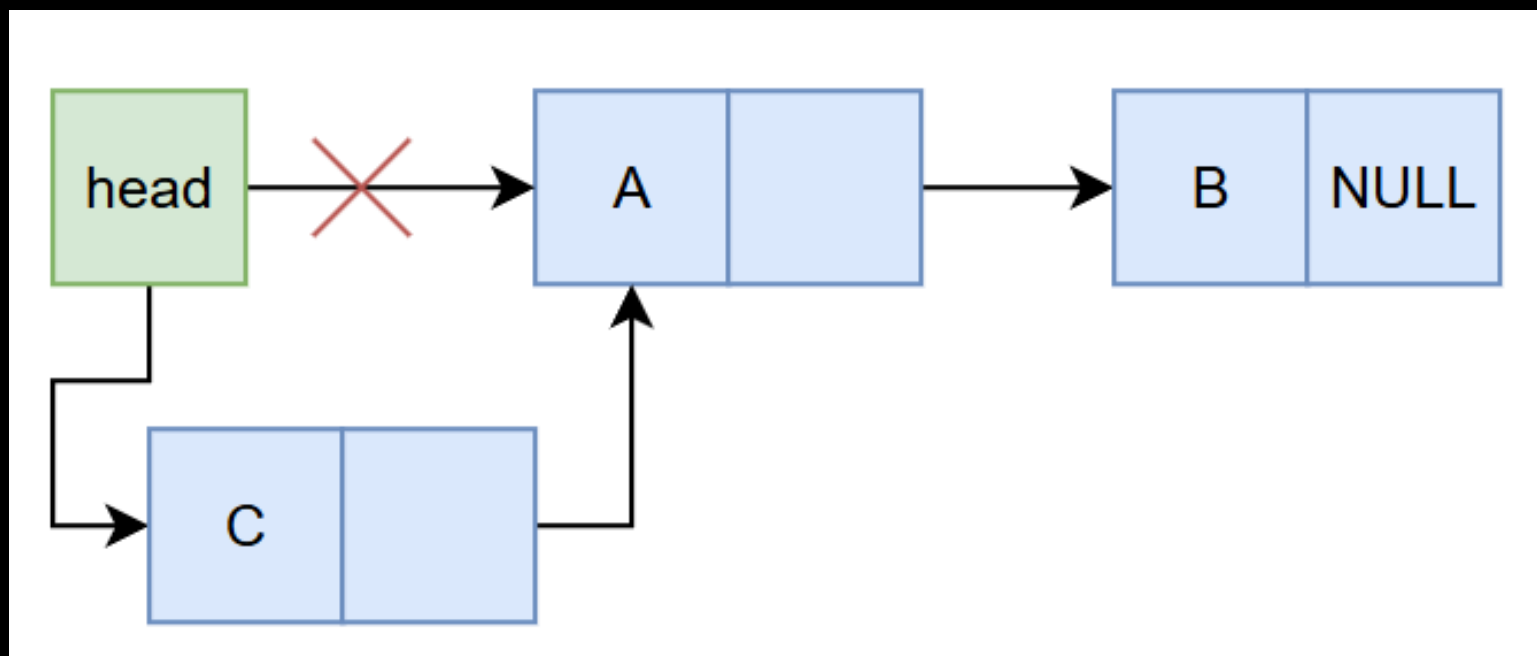
Поиск в списке

```
// Функция поиска, проходим по списку,  
// если элемент в списке вернем true иначе false  
bool searchList(node_t* head, int target);
```

- Пока текущий узел не равен NULL:
 - Если data = искомой
 - Возвращаем true (искомый элемент есть в списке)
 - Приравниваем текущий указатель на следующий

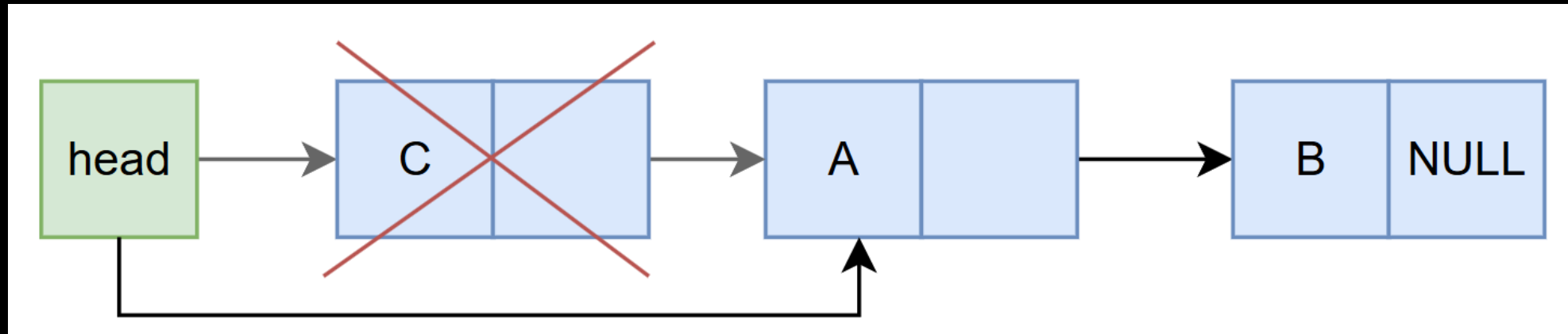
Добавление элемента в начало

- Создать новый узел с заданным значением.
- Установить следующий указатель нового узла на текущий head.
- head должен указывать на новый узел.
- Вернуть новую голову списка.



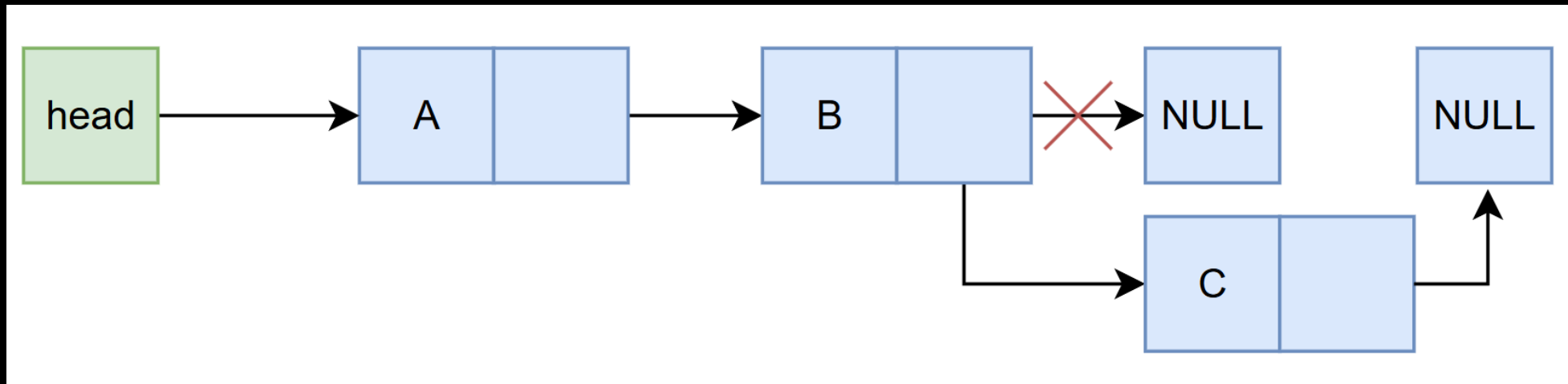
Удаление первого узла

- Проверить, имеет ли голова значение NULL
 - Если да, то вернуть NULL (список пуст)
- Сохранить текущий head во временной переменной
- Переместить указатель головы на следующий узел
- Удалить временный узел
- Вернуть новый head связанного списка



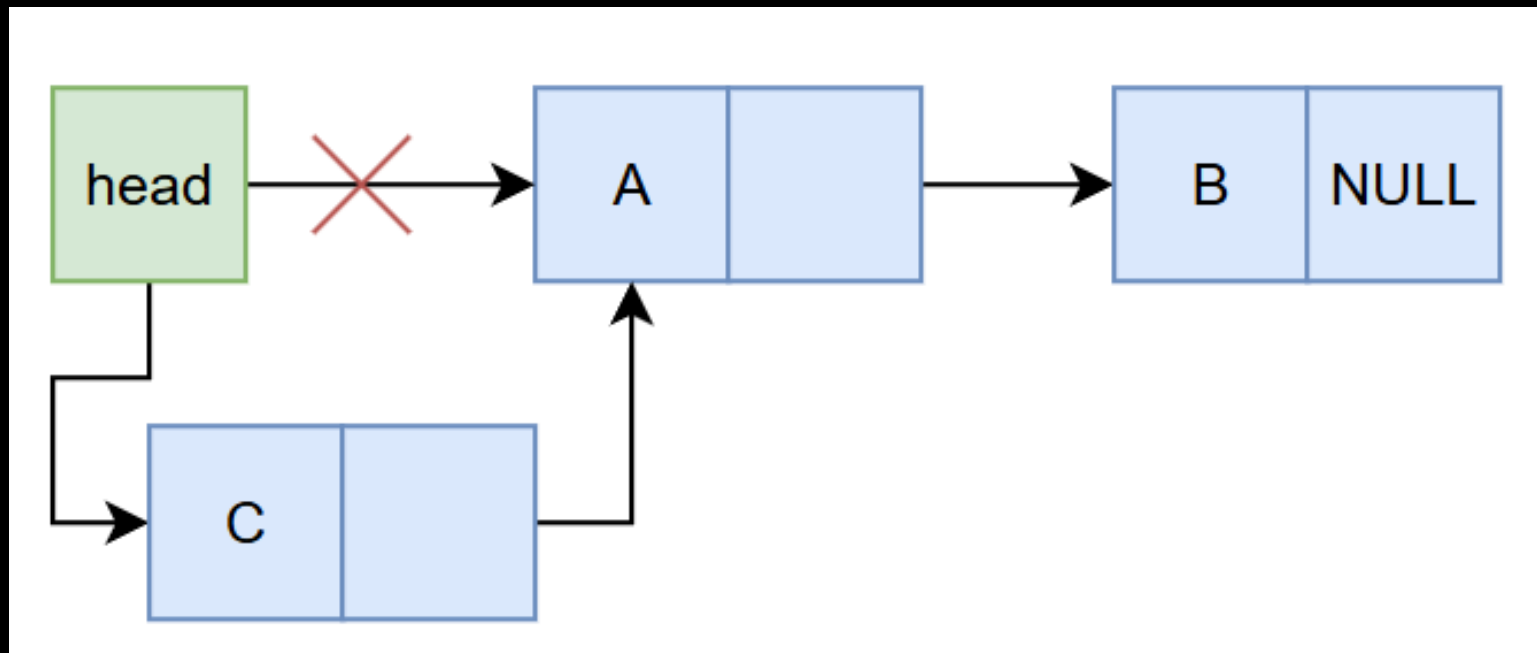
Добавление элемента в конец

- Создать новый узел с заданным значением.
- Проверить пуст ли список:
 - Если да, сделать новый узел головой и вернуть
- Перемещаться по списку до тех пор, пока не будет достигнут последний узел
- Связать новый узел с текущим последним узлом, установив указатель следующего узла у последнего узла на новый узел



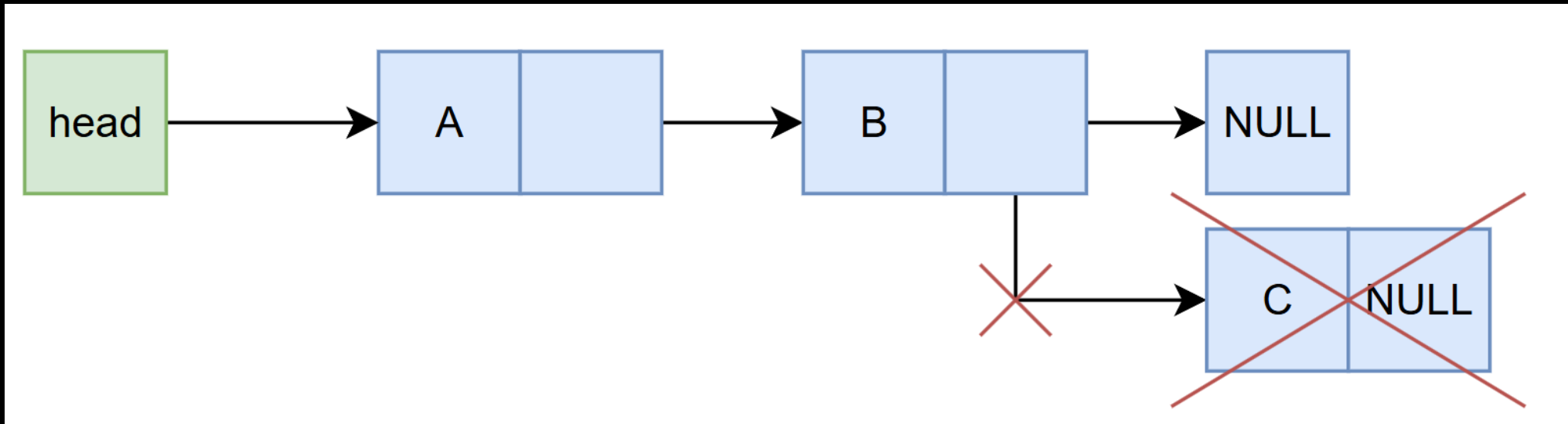
Добавление элемента в начало списка

- Создать новый узел с заданным значением.
- Установить следующий указатель нового узла на текущий head.
- head должен указывать на новый узел.
- Вернуть новую голову списка.



Удаление последнего узла

- Проверить пуст ли список (head = NULL)
 - Если да, вернуть NULL (список пуст)
- Проверить, равен ли следующий узел от головы NULL (только один узел в списке - голова)
 - Если да, удалить head и вернуть NULL
- Пройти по списку, чтобы найти предпоследний узел (второй с конца)
- Удалить последний узел (узел после предпоследнего)
- Установить следующий указатель предпоследнего узла в NULL
- Возвращаем head списка



Вставка в середину и удаление из середины

Здесь я уже устал рисовать и расписывать - поэтому самостоятельно.

Что для этого нужно сделать?

Понять все примеры выше с поиском и вставкой/удалением, а также не забывать вначале проверку на пустой список, чтоб не схватить сегу (segfault)

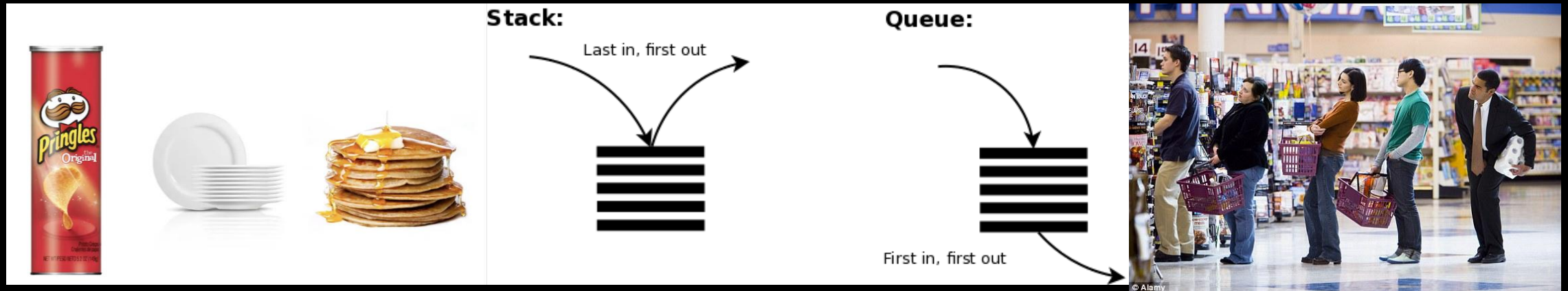


Стек и Очередь

Стек (англ. stack - стопка) - еще одна из простейших структур данных. Для описания стека часто используется аббревиатура LIFO (Last In, First Out), подчёркивающая, что элемент, попавший в стек последним, первым будет из него извлечён. Классический стек поддерживает всего лишь три операции:

- Добавить элемент в стек `push()` (Сложность: $O(1)$)
- Извлечь элемент из стека `pop()` (Сложность: $O(1)$)
- Проверить, пуст ли стек `isEmpty()` (Сложность: $O(1)$)

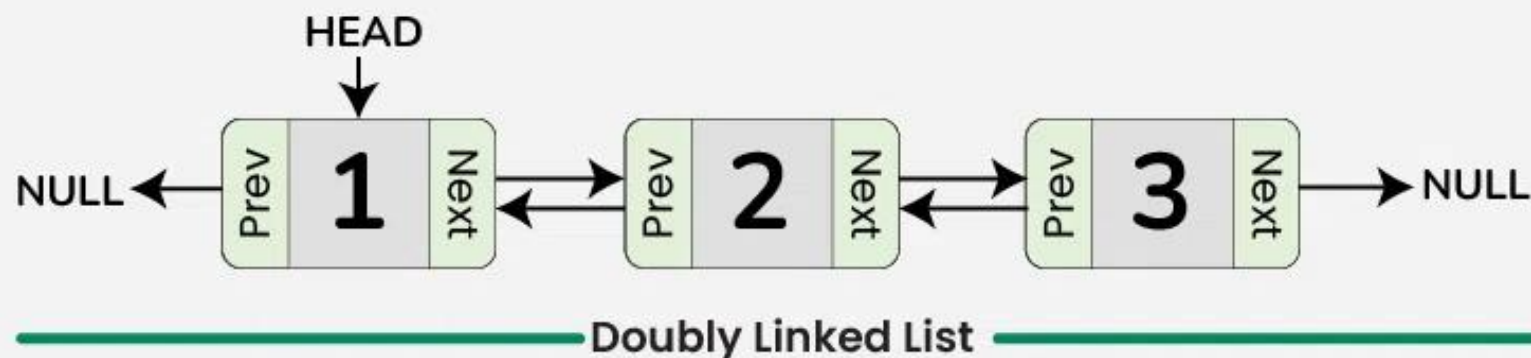
Очередь (англ. queue) поддерживает тот же набор операций, что и стек, но имеет противоположную семантику. Для описания очереди используется аббревиатура FIFO (First In, First Out), так как первым из очереди извлекается элемент, раньше всех в неё добавленный.



Двусвязный список

src/doubly_ll.c

```
node_t {  
    node_t *prev;  
    int data;  
    node_t *next;  
};
```

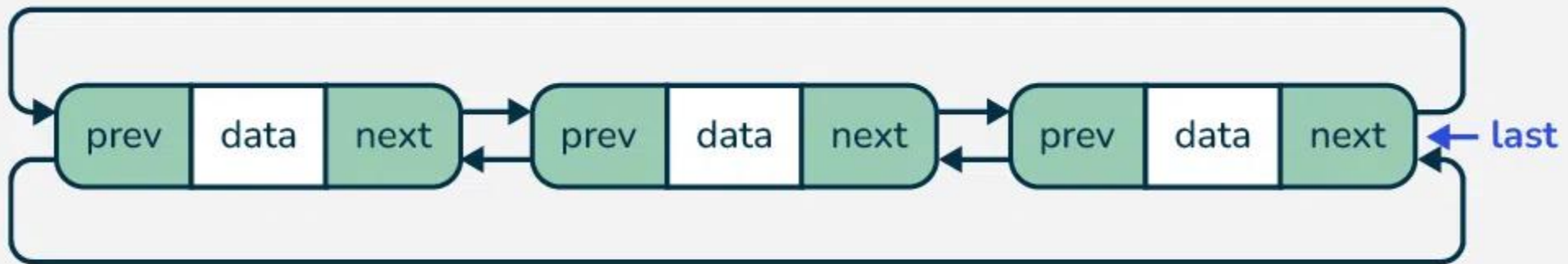


- Кнопки "Назад" и "Вперед" в браузере
- "Плейлист в плеере"/видосы/рилсы
- Ctrl+Z и Ctrl+Y (Отмена/Повтор)
- Алгоритмы кэширования (например, LRU)
- Дек (Deque - двусторонняя очередь), для эффективной вставки и удаления на обоих концах

Циклический список



Representation of circular linked list



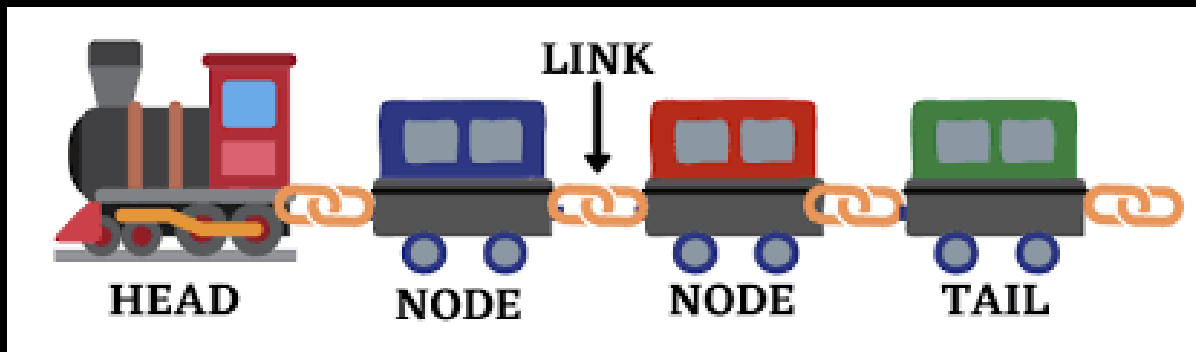
Representation of circular doubly linked list

*Как понять является список зацикленным?

Итого по спискам

Связанный список — это линейная структура данных, которая используется для хранения набора данных с помощью узлов

- Последовательные элементы соединены указателями/ссылками.
- Последний узел связанного списка указывает на null.
- Точка входа связанного списка называется головой.
- Обычные вариации связанных списков — это Singly, Doubly, Singly Circular и Doubly Circular.



Преимущества связанных списков (или наиболее распространенные варианты использования):

- Связанные списки в основном используются из-за их эффективной вставки и удаления. Нам нужно изменить всего несколько указателей (или ссылок), чтобы вставить (или удалить) элемент в середине
- Вставка и удаление в любой точке связанного списка занимает $O(1)$ времени. Тогда как в структуре данных массива вставка/удаление в середине занимает $O(n)$ времени.
- Эта структура данных проста и может также использоваться для реализации **стека, очередей и других абстрактных структур данных**. Большинство языковых библиотек используют связанный список внутренне для реализации этих структур данных.
- Связанный список может оказаться более эффективным по сравнению с массивами в случаях, когда мы не можем заранее угадать количество элементов. В случае массивов вся память для элементов выделяется вместе. Даже с динамическими массивами, такими как `vector` в C++ или `list` в Python или `ArrayList` в Java, внутренняя работа включает в себя освобождение всей памяти и выделение большего фрагмента, когда вставки происходят за пределами текущей емкости.

Недостатки связанных списков

Связанные списки являются популярной структурой данных, но, как и любая другая структура данных, они имеют определенные недостатки. Некоторые из основных недостатков связанных списков:

- Медленное время доступа: доступ к элементам в связанном списке может быть медленным, так как вам нужно пройти по связанному списку, чтобы найти нужный элемент, что является операцией $O(n)$. Это делает связанные списки плохим выбором для ситуаций, когда вам нужно быстро получить доступ к элементам.
- Указатели или ссылки: Связанные списки используют указатели или ссылки для доступа к следующему узлу, что может сделать их более сложными для понимания и использования по сравнению с массивами. Эта сложность может сделать связанные списки более трудными для отладки и обслуживания.
- Более высокие накладные расходы: Связанные списки имеют более высокие накладные расходы по сравнению с массивами, так как каждый узел в связанном списке требует дополнительной памяти для хранения ссылки на следующий узел.
- Неэффективность кэширования: Связанные списки неэффективны в отношении кэширования, так как память не является непрерывной. Это означает, что при обходе связанного списка вы вряд ли получите нужные вам данные в кэше, что приведет к промахам кэша и низкой производительности.

Примеры применения связанных списков в реальном мире

- В веб-браузере URL-адреса предыдущей и следующей веб-страницы могут быть связаны с помощью кнопок «предыдущая» и «следующая» (двухсвязный список)
- В средстве просмотра изображений предыдущее и следующее изображения могут быть связаны с помощью кнопок «предыдущая» и «следующая» (двухсвязный список)
- Циклические связанные списки могут использоваться для реализации действий в циклическом режиме, когда мы переходим к каждому элементу по одному через время t
- Связанные списки предпочтительнее массивов для реализации структур данных Queue и Deque из-за быстрых удалений (или вставок) из начала связанных списков.

<https://github.com/kruffka/C-Programming>

